PRACTICAL ASSESSMENT · SENIOR .NET ENGINEER

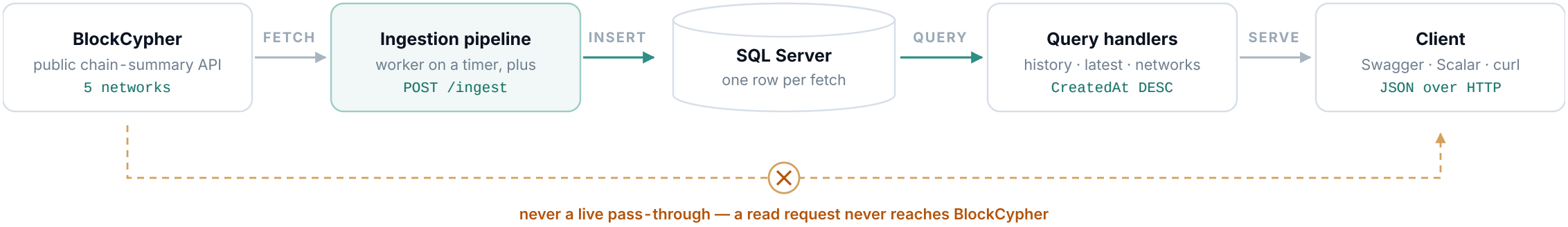
BlockCypher Snapshot API

A .NET 10 Web API that ingests network summaries for five blockchains, stores every response with its fetch timestamp, and serves the accumulated history newest-first.

RUNTIME	ARCHITECTURE	STORAGE	RUN PATH
.NET 10 (LTS) · C# 14	Clean + Vertical Slice + CQRS	SQL Server 2022 · EF Core 10	docker compose up --build

THE SOLUTION IN ONE PICTURE

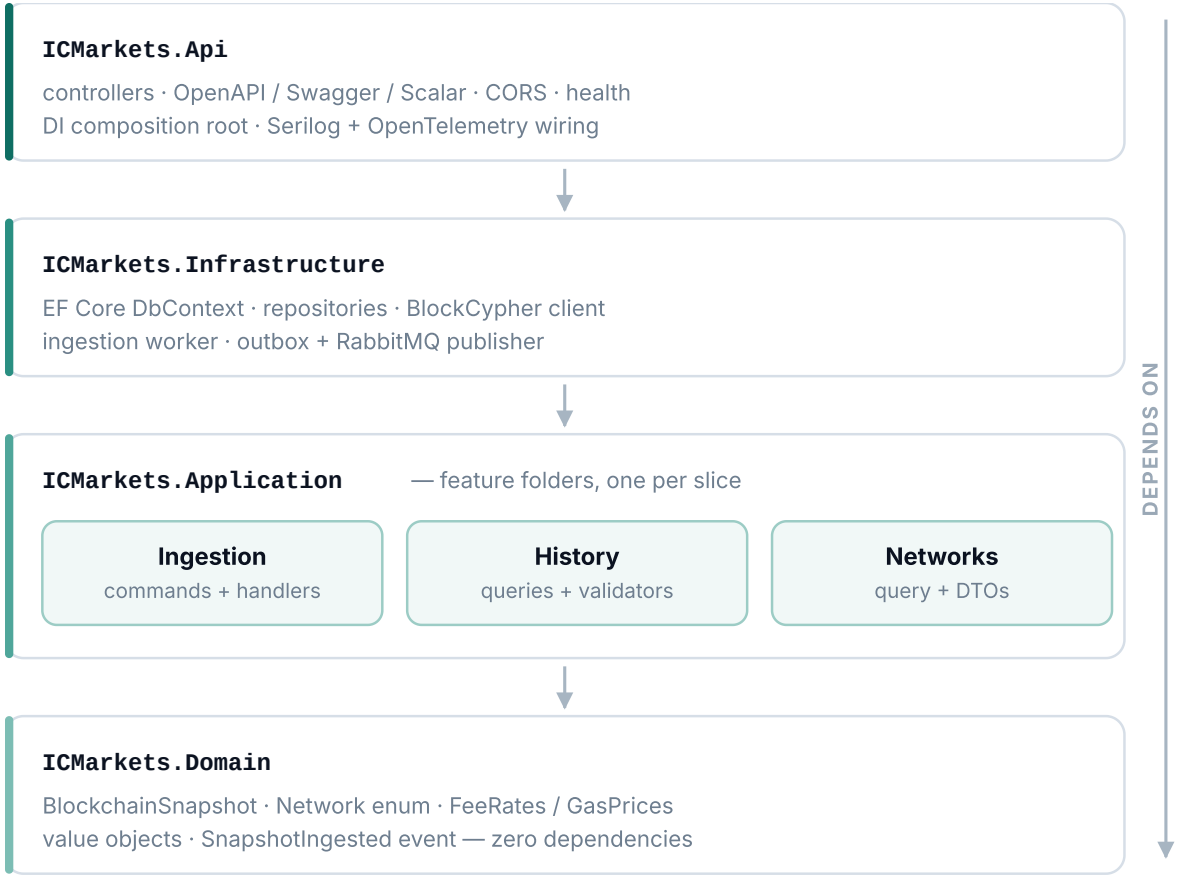
Collect on a timer, store immutably, serve from the database



ENDPOINT	BEHAVIOUR
`POST /api/v1/blockchains/ingest`	Fetches all five networks now. Per-network failure isolation — a failing chain is reported in `failed`, the batch still commits the rest.
`GET …/{network}/history`	Stored snapshots for one network, newest first, `take`/`skip` paged and index-backed.
`GET …/{network}/latest`	Most recent stored snapshot; `404` before the first ingestion.
`GET /health · /metrics`	Liveness with a DB connectivity probe; Prometheus scrape surface.

ARCHITECTURE

Clean layers, with vertical feature slices inside Application



THREE PATTERNS, NAMED ON PURPOSE

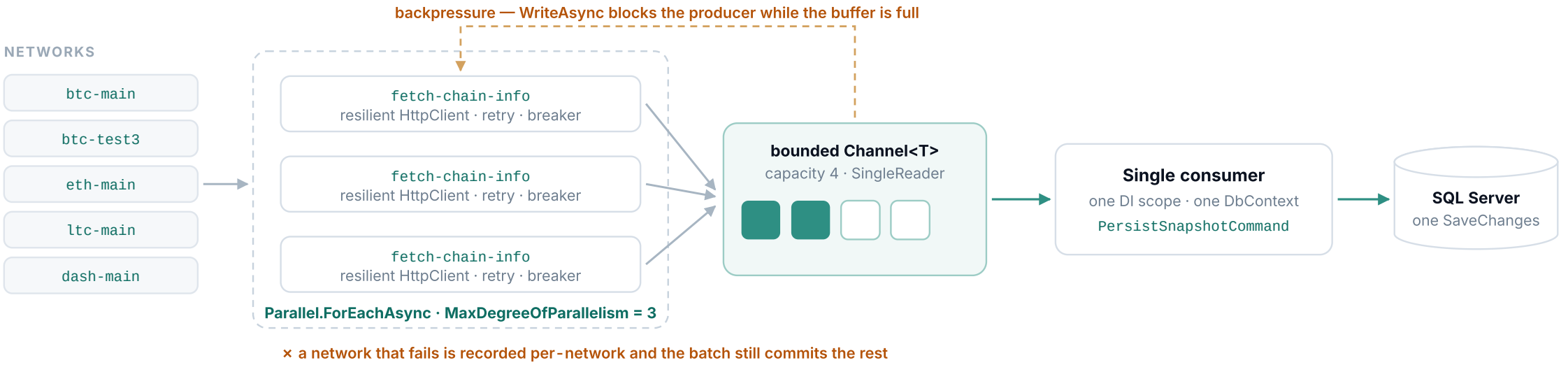
Repository — `IBlockchainRepository` stages changes and never calls `SaveChanges` itself.

Unit of Work — `AppDbContext` implements `IUnitOfWork`, so one handler commits several repository calls atomically.

CQRS — MediatR commands/queries per feature; the validation pipeline behaviour hangs off the same pipeline, so request validation is automatic rather than per-controller.

Why the boundary is real, not cosmetic. The database provider is a configuration switch (`Database:Provider` = SqlServer | Sqlite). Swapping it changes no Application or Domain code — which is the only honest proof that the layering does something.

A bounded producer/consumer pipeline, not a fan-out



- **Thread safety by construction.** `DbContext` is not thread-safe, so every branch resolves its own scope via `IServiceScopeFactory` instead of sharing one.
- **Cancellation end to end.** The token flows from the host into HTTP and EF; the writer is completed in a `finally`, so the consumer loop can never hang.
- **Failure isolation.** A failed fetch or persist is captured per network into a `ConcurrentBag`; the caller gets the successes and the failures back in one result.
- **Resilience at the edge.** `IHttpClientFactory` + Microsoft.Extensions.Http.Resilience (Polly): timeout, retry with jitter, circuit breaker, `429 Retry-After` honoured.

PLINQ was rejected deliberately. It partitions CPU-bound work across threads; five HTTP fetches are I/O-bound, so those threads would simply sit on sockets. Bounded concurrency also respects BlockCypher's rate limiter, where an unbounded fan-out would trip it.

DATA MODEL & THE HISTORY QUERY

Immutable snapshots, deterministic ordering, an index that matches

BlockchainSnapshot

Id
Network · nvarchar(450), index key
Height · Hash · Time
PeerCount · UnconfirmedCount · forks
RawJson · the response as provided
CreatedAt · UTC, from TimeProvider

`one row per fetch, never updated`

owns exactly one of

FeeRates
high / medium / low fee per kb
`BTC · LTC · Dash`

mutually exclusive

GasPrices
high / medium / low gas price
`ETH`

```
await context.Snapshots.AsNoTracking()
    .Where(s => s.Network == network)
    .OrderByDescending(s => s.CreatedAt)
    .ThenByDescending(s => s.Id)   // deterministic
    .Skip(skip).Take(take).ToListAsync(ct);
```

The ordering and the index were designed together: the query above is exactly the access pattern `(Network, CreatedAt DESC)` was created for.

ORDERING, DONE PROPERLY

History orders by `CreatedAt DESC` with `Id DESC` as a tiebreaker for rows sharing a timestamp, backed by a composite `(Network, CreatedAt DESC)` index: filter by network, then scan newest-first. On SQL Server the nonclustered index implicitly carries the clustered key, so the tiebreaker is covered without a third column.

READS

`AsNoTracking()` projections, server-side paging, and `EnableRetryOnFailure` on the SQL Server provider so a transient connection fault retries instead of surfacing as a 500.

A limitation found and handled, not hidden. EF Core's SQLite provider cannot translate `ORDER BY` over `DateTimeOffset`. Since SQLite is the no-Docker fallback and the functional-test database, `CreatedAt` gets a provider-scoped conversion to UTC ticks there; SQL Server keeps its native type. A design-time context factory pins `dotnet ef` to SQL Server so migrations stay provider-correct.

TECHNOLOGIES, BY THE CONCERN THEY SERVE

Every dependency earns its place

PLATFORM

.NET 10 (LTS), C# 14, ASP.NET Core Web API with controllers. Central Package Management pins every version in one file.

PERSISTENCE

EF Core 10 + **SQL Server 2022** (pinned image), migrations applied on startup; **SQLite** as a config-selected fallback provider.

API SURFACE

Native **OpenAPI 3.1** (`Microsoft.AspNetCore.OpenApi`) with both **Swagger UI** and **Scalar** on top; RFC 7807 ProblemDetails.

APPLICATION PATTERNS

MediatR 12.5.0 for CQRS, **FluentValidation** via a pipeline behaviour, **Mapperly** source-generated mapping — no runtime reflection.

CONCURRENCY & RESILIENCE

System.Threading.Channels, `Parallel.ForEachAsync`, **IHttpClientFactory** + **Http.Resilience** (Polly), injected **TimeProvider**.

MESSAGING

Transactional **outbox** publishing `SnapshotIngested` to a **RabbitMQ** fanout exchange behind an `IEventPublisher` seam.

OBSERVABILITY

OpenTelemetry traces/metrics/logs with custom ingestion spans and meters, **Prometheus** scrape endpoint, provisioned **Grafana** dashboard, **Serilog** in Elastic Common Schema.

TESTING

xUnit, **NSubstitute**, **AwesomeAssertions**, **Testcontainers.MsSql**, **WireMock.Net**, `WebApplicationFactory`, **BenchmarkDotNet**.

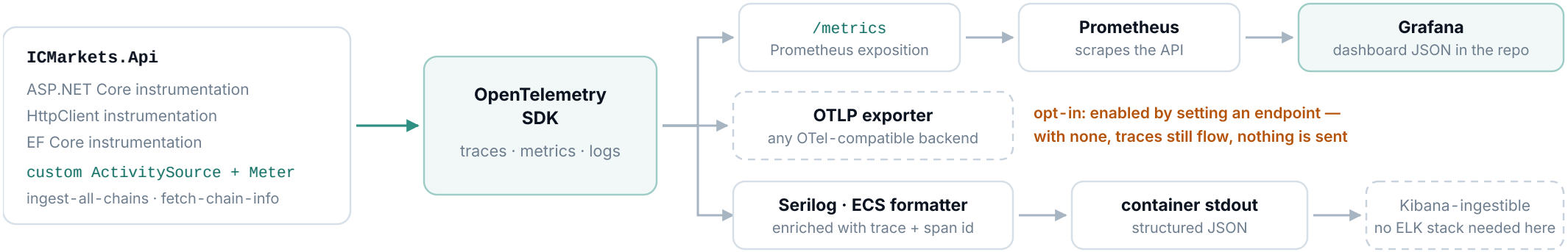
RUNTIME & DELIVERY

Multi-stage **Docker** images, **Docker Compose** as the verified path, **YARP** gateway, **Kubernetes** manifests, **GitHub Actions** CI.

Licensing was a selection criterion, not an afterthought. AutoMapper 15+, MediatR 13+, FluentAssertions 8+ and NBomber all moved to commercial or personal-use-only terms in 2025–2026. Each was replaced with a permissively licensed equivalent, and Central Package Management makes a silent drift onto a paid version impossible.

OBSERVABILITY

Domain telemetry, not just framework defaults

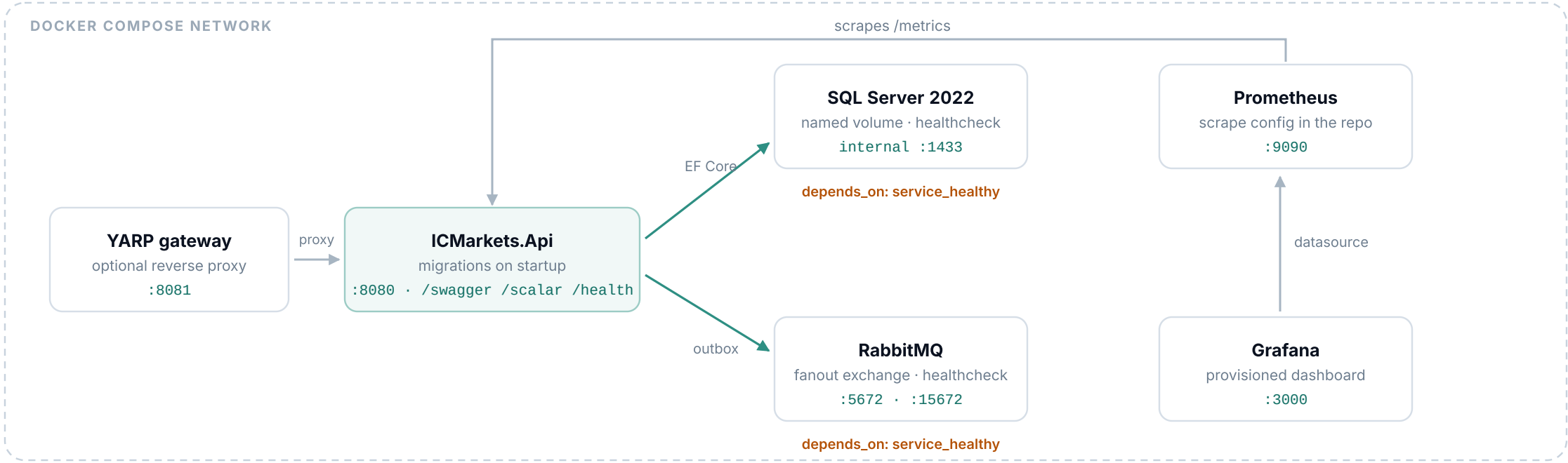


- **Logs correlate to traces.** Every line carries the active trace and span id, so a log entry leads back to the trace that produced it.
- **Domain metrics, not just framework counters** — snapshots per network, fetch-duration histogram, failure counter.
- **Health is a real readiness signal** — `/health` includes a database connectivity check.
- **Nothing to configure** — the dashboard is provisioned in the repository and fills in on the first ingestion.

Why this is a core deliverable and not a bonus. OpenTelemetry appears in both halves of the role description. It is also the highest-leverage addition to an otherwise ordinary CRUD assessment: a reviewer who runs one command and watches ingestion metrics move remembers the submission.

RUNTIME TOPOLOGY

One command brings up the whole stack



Two startup mechanisms, because they solve different problems. The compose healthcheck plus `depends_on: service_healthy` stops the API starting before the database accepts connections; `EnableRetryOnFailure` handles transient faults later, at runtime, which a healthcheck cannot.

The same shape in Kubernetes. `k8s/` carries Deployment, Service and ConfigMap manifests for this stack; compose remains the verified path. A local run without Docker drops to the SQLite provider and needs none of this.

TESTING

Four projects, each testing something the others cannot

25
Unit test cases — handlers, validators, mapping, BlockCypher JSON deserialization

11
Integration tests against a **real SQL Server** container via Testcontainers

7
Functional tests over the full HTTP pipeline with BlockCypher stubbed

2
BenchmarkDotNet suites on the mapping and query-handler paths

WHAT EACH SUITE EXERCISES	Unit	Integration	Functional	Performance
HTTP pipeline, controllers, DI	—	—	`real`	—
Command / query handlers	`real`	—	`real`	`real`
Repository + EF Core	`in-memory fake`	`real`	`real`	`in-memory fake`
Database engine	—	`SQL Server · Testcontainers`	`SQLite in-memory`	—
BlockCypher upstream	`NSubstitute`	—	`WireMock.Net`	—

the real component runs · substituted on purpose · not exercised by that suite

Benchmarks: mapping ~34 ns / 224 B · fifty-row history projection ~1.9 µs

Determinism over luck. `TimeProvider` is injected everywhere `CreatedAt` is set, so ordering assertions test the query rather than racing the system clock. The deserialization tests run against payloads captured from the live API, not hand-written JSON.

Why not the EF in-memory provider. It does not enforce relational behaviour, so it hides exactly the bugs integration tests exist to catch. **CI:** GitHub Actions restores, builds with warnings-as-errors and runs all three suites on every push to `main` and `development`.

DECISIONS WORTH DEFENDING

Where the obvious answer was the wrong one

DECISION	REASONING
.NET 10, not .NET 8	The brief allows .NET 6+, but .NET 8 and 9 both reach end of support on 10 November 2026. Shipping new work onto a runtime that is EOL within days is the wrong default. Risk accepted and mitigated: Docker is the primary documented run path, verified from a clean clone.
SQL Server 2022, pinned — with a SQLite fallback	SQL Server is named in the role and matches the shop. The `2022-latest` image is pinned rather than 2025, whose image has a reported system-database defect — a broken container on a reviewer's machine costs more than a newer major gains. SQLite stays as the no-Docker path and the functional-test database.
Channels + bounded parallelism, not PLINQ	Five HTTP calls are I/O-bound; PLINQ partitions CPU-bound work. Bounded concurrency also respects BlockCypher's rate limiter, where an unbounded fan-out would trip it.
Permissive dependencies only	AutoMapper → **Mapperly**, MediatR 13+ → **MediatR 12.5.0 pinned**, FluentAssertions 8+ → **AwesomeAssertions**, Moq → **NSubstitute**, NBomber → **BenchmarkDotNet**. Each replacement is a licence decision with a cost attached at ICMarkets' scale, enforced by Central Package Management.
DDD vocabulary, scoped short	A rich entity with a static factory, two value objects and a domain event — enough to show the habits. Full tactical DDD on five snapshot tables would be over-engineering, and knowing where to stop is part of the answer.
Stored history, never a live pass-through	Reads answer from the database, so `CreatedAt` means “when it was fetched” and history is real rather than synthesised per request. It also makes the functional suite independent of BlockCypher's availability and rate limits.

SCOPE, HONESTLY

What is deliberately out, and where it would go next

BEYOND THE BRIEF, ALREADY IN

Full OpenTelemetry, Prometheus and a provisioned Grafana dashboard

Transactional outbox and RabbitMQ event publishing

YARP API gateway and Kubernetes manifests

GitHub Actions CI and BenchmarkDotNet performance tests

Five ADRs plus a decision log covering every non-obvious choice

Core requirements first. The brief is scored directly; the additions above only amplify it. Nothing optional was allowed to put a required capability at risk.

REVIEWING IT IN FIVE MINUTES

1. `docker compose up --build`, then open `:8080/swagger`.
2. Call `POST /ingest` once, then `GET …/btc-main/history` — and watch the Grafana dashboard on `:3000` fill in.
3. `dotnet test` for the three suites; `docs/adr/` for the reasoning behind every choice above.

NOT IMPLEMENTED	WHY / NEXT STEP
Authentication	Out of the brief's scope. A real deployment sits behind JWT/OAuth2 with the CORS policy scoped to actual origins.
Distributed cache	`latest` hits the database per call; a short-TTL Redis read-through would cut load without touching ingestion correctness.
Kafka / SQS	`IEventPublisher` is the seam — a new implementation, not a redesign. RabbitMQ was the cheapest real broker to run in compose.
Saga pattern	One aggregate, no cross-service transaction to coordinate. It becomes relevant if ingestion and reads split into separate services.
Hardened docs exposure	Swagger and Scalar are reachable in every environment so a reviewer needs no environment switch; production would gate them behind a flag.